Lesson 5 — Analog to Digital Conversion (ADC)

ATmega328P / Arduino Uno | Source: lecture 05 slides, Exercise 4, "1023 or 1024" note, LM35 / LDR datasheets | Builds on Lessons 3 and 4

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Explain why an ADC is needed and how it turns a voltage into a number.
- Calculate resolution, interval size, the ADC value for a voltage and the voltage for an ADC value.
- Configure `ADMUX` and `ADCSRA` in C, read with polling and with the ADC interrupt, and use `analogRead`.
- Use a potentiometer, an LDR (voltage divider) and an LM35 as analog inputs.
- Design the Exercise 4 "auto light", including the mains extension.

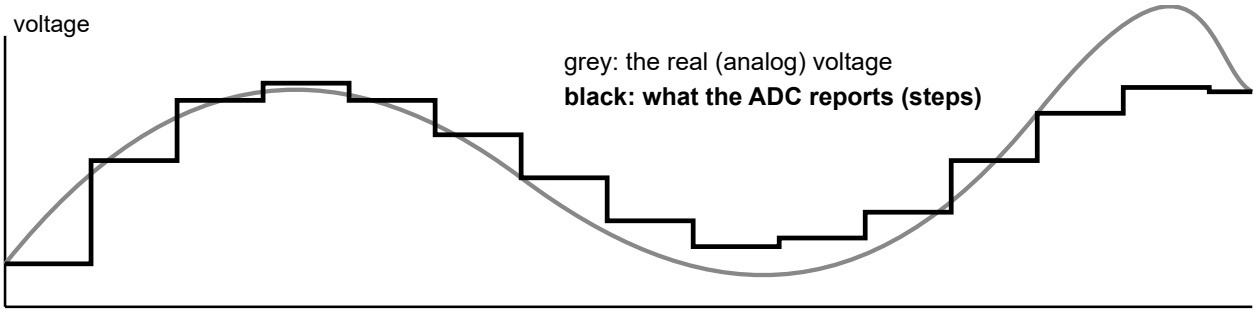
GIVEN TO YOU IN THE EXAM (DO NOT MEMORISE THESE)

ATmega328P Simplified Data Sheet: the bits of `ADMUX` (REFS1, REFS0, ADLAR, MUX3..0), `ADCSRA` (ADEN, ADSC, ADIF, ADIE, ADPS2..0), `ADCL` / `ADCH`, the **REFS table** (01 = AVcc, 11 = 1.1 V internal) and the **ADPS prescaler table** (111 = F_CPU/128), plus `ADC_vect`. **Arduino cheat sheet**: `analogRead(pin)`, `analogReference(DEFAULT / INTERNAL / EXTERNAL)`, pins A0–A5, `map()`.

Not given, so learn them here: what each bit is *for*, the formulas (1024 not 1023), the order of setup steps, why /128, the voltage divider, the LM35 scale, and how to calibrate thresholds.

1. Why an ADC? Analog vs digital

The physical world is **analog**: temperature, light and position change smoothly and can take any value. A sensor turns that quantity into a **voltage** that also changes smoothly. The CPU only understands **numbers** (digital). The **Analog to Digital Converter** measures the voltage on a pin and gives the CPU a whole number for it.



The ADC takes a reading (sample) and rounds it to one of 1024 levels. Finer levels = closer to the real curve.

Figure 1. Converting means sampling and rounding. The size of a step is the ADC's resolution.

2. The key idea: 1024 intervals between 0 V and Vref

Term	Meaning (ATmega328P values)
Resolution	Number of bits in the result. Ours is **10-bit**.
Vref (reference)	The top of the measuring range. The ADC measures 0 V up to (just below) Vref. Default Vref = AVcc = **5 V**.
Number of intervals	$2^{\text{resolution}} = 2^{10} =$ **1024**, so the result is a number from **0 to 1023**.
Interval size ΔV (step)	Vref / 1024 = 5 / 1024 = **4.88 mV**. The smallest voltage change the ADC can notice.

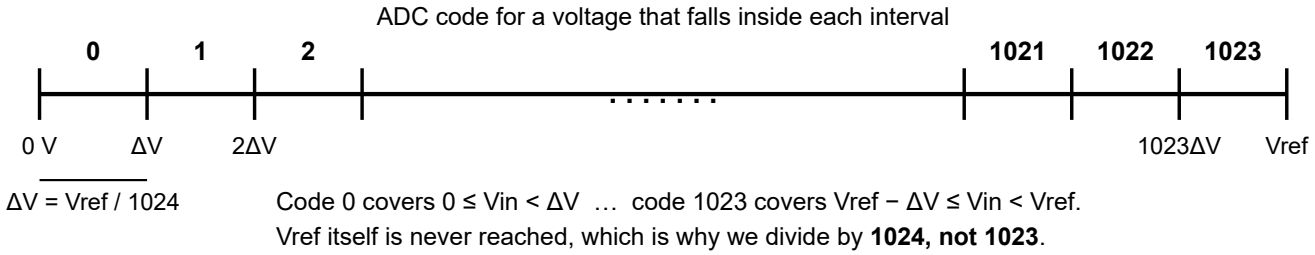


Figure 2. The range 0 to Vref is cut into 1024 equal intervals; the ADC tells you which interval Vin is in.

THE TWO FORMULAS (LEARN BOTH DIRECTIONS)

ADC = Vin × 1024 / Vref (round down) **Vin = ADC × Vref / 1024**

Examples (Vref = 5 V): Vin = 2.5 V → ADC = 2.5 × 1024 / 5 = **512**. ADC = 300 → Vin = 300 × 5 / 1024 = **1.465 V**. ADC = 1023 → 4.995 V (the top interval).

Why 1024? Each of the 1024 codes is an interval of width Vref/1024. Code 1023 is the last interval, just under Vref, the same way a clock's "23" hour is not the end of the day. Using 1023 is a small (0.1%) but real error.

Getting a more accurate reading

The real voltage can be anywhere inside its interval, so the reading is uncertain by up to ΔV. To make ΔV smaller: **(1) use more bits** (fixed at 10 on this chip), or **(2) lower Vref** to just above the largest voltage you expect.

Signal range	Choose Vref	ΔV	How
0 – 5 V (pot, LDR divider)	AVcc = 5 V	4.88 mV	REFS1:0 = 01, `analogReference(DEFAULT)`
0 – 1 V (e.g. LM35 up to 100 °C)	internal 1.1 V	1.07 mV	REFS1:0 = 11, `analogReference(INTERNAL)`
0 – 2.5 V	2.5 V supplied on the AREF pin	2.44 mV	REFS1:0 = 00, `analogReference(EXTERNAL)`

Warning: a voltage above Vref just reads 1023 (the ADC saturates). Never put more than 5 V on an analog pin. Never connect a voltage to AREF while REFS selects an internal reference.

3. Inside the ADC: how it finds the number

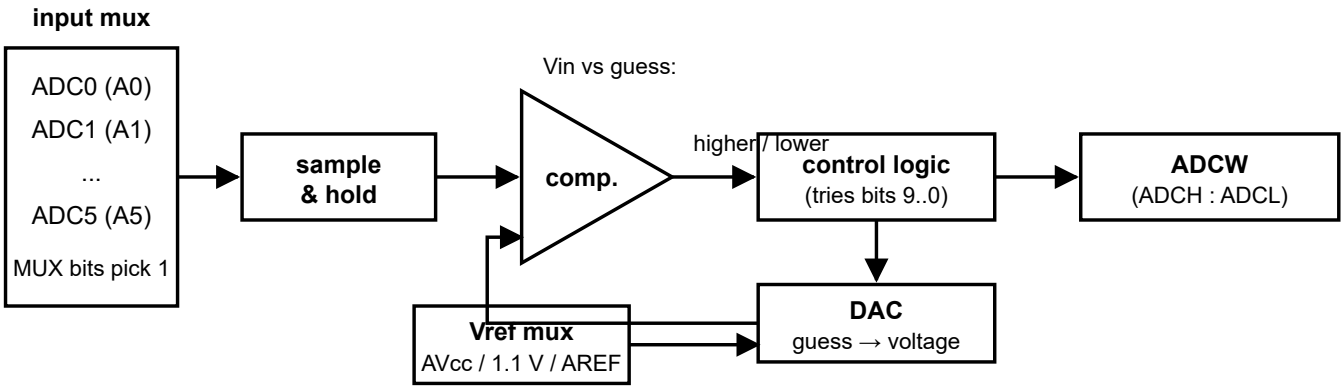


Figure 3. Successive approximation: the logic makes a guess, the DAC turns the guess into a voltage, the comparator says "Vin is higher" or "lower", and the logic improves the guess one bit at a time.

Like guessing a number 0–1023 with yes/no questions. Start at the middle (512): is Vin above it? Keep that bit, halve the step, ask again. 10 questions find the exact interval. Worked example with Vin = 3.2 V (should give 655):

step	try bit	guess	guess V	3.2 V ≥ guess?
1	9 (512)	512	2.500	yes, keep
2	8 (256)	768	3.750	no, drop
3	7 (128)	640	3.125	yes, keep
4	6 (64)	704	3.438	no
5	5 (32)	672	3.281	no
6	4 (16)	656	3.203	no
7	3 (8)	648	3.164	yes
8–10	2, 1, 0	652, 654, 655	…	yes, yes, yes

Result 655 = 3.2 × 1024 / 5 = 655.36 rounded down. ✓

WHY THE ADC CLOCK IS F_CPU / 128

The ADC needs its own slower clock: **50–200 kHz** for full 10-bit accuracy. The prescaler divides the CPU clock: 16 MHz / 128 = **125 kHz** (in range). /64 would give 250 kHz, too fast. ADPS2:0 = 111 selects /128.

HOW LONG A CONVERSION TAKES

About **13 ADC clocks**: 13 / 125 kHz ≈ **104 µs** (the very first conversion after enabling takes 25 clocks). So about 9,600 readings per second, fast for buttons, too slow for audio.

SAMPLE & HOLD

The input voltage is copied onto a tiny capacitor at the start, then held steady while the 10 comparisons run, so a changing signal does not confuse the search.

4. The ADC registers

Register / bits	What it does	Typical setting
ADMUX: REFS1, REFS0	Choose Vref (table on exam sheet).	01 = AVcc (5 V)
ADMUX: ADLAR	0 = result right-adjusted (normal). 1 = left-adjusted (see below).	0
ADMUX: MUX3..0	Which pin to measure: 0000 = ADC0 (A0) … 0101 = ADC5 (A5). Change only when no conversion is running.	0000 = A0
ADCSRA: ADEN	**Enable** the ADC (it is off after reset to save power).	1
ADCSRA: ADSC	Write 1 to **start** a conversion. It **reads 1 while converting and goes back to 0 by itself when finished**: that is how you poll.	1 to start
ADCSRA: ADIF	Flag set when a conversion finishes and the result is in the data registers.	(hardware)
ADCSRA: ADIE	Enable the "conversion complete" interrupt (`ADC_vect`).	1 for interrupts
ADCSRA: ADPS2..0	ADC clock prescaler (table on exam sheet).	111 = /128
ADCL, ADCH, ADCW	The 10-bit result. `ADCW` reads both as one 16-bit number (6 leading 0s + 10 result bits). Reading ADCL alone first locks ADCH until it is read, so just use `ADCW`.	read only

ADLAR = 0 (right-adjusted): read `ADCW` = 0 to 1023

ADCH (bits 15–8)	0	0	0	0	0	0	9	8
ADCL (bits 7–0)	7	6	5	4	3	2	1	0

numbers = result bit held in that box

ADLAR = 1 (left-adjusted): read only `ADCH` = the top 8 bits = 0 to 255

ADCH	9	8	7	6	5	4	3	2
ADCL	1	0	0	0	0	0	0	0

use when 8-bit accuracy is enough (then Vin = ADCH × Vref / 256)

5. System 8: `voltMeter` (polling)

Task: measure 0–5 V from a **10 kΩ potentiometer** on **PC0 / ADC0 (A0)** and show it as a bar of **5 LEDs** on **PB0–PB4** (pins 8–12). Above 0 V: 1 LED, above 1 V: 2 LEDs … above 4 V: 5 LEDs.

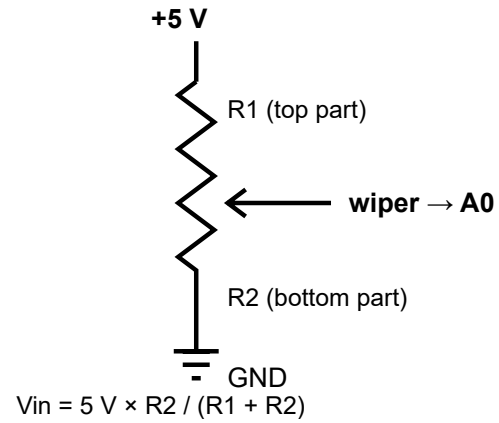


Figure 4. A potentiometer is a voltage divider you can turn: the wiper picks a fraction of 5 V.

Voltage divider: two resistors in series between 5 V and GND. The voltage at the middle is **Vout = Vs × R2 / (R1 + R2)**, where R2 is the resistor to GND. Turning the knob moves the wiper, changing R1 and R2 (they always add up to 10 kΩ), so Vin goes smoothly from 0 to 5 V. This is the standard way to turn a **changing resistance** (pot, LDR, thermistor) into a **voltage** the ADC can read.

Example: wiper at 30% from the bottom: R2 = 3 kΩ, R1 = 7 kΩ, Vin = 5 × 3 / 10 = 1.5 V, ADC = 307, LEDs: 2.

```
#include <avr/io.h>

int main(void)
{
    DDRB = DDRB | 0b00011111;                   // PB0-PB4 outputs (LEDs)
    DDRC = DDRC & 0b11111110;                   // PC0 (ADC0) input
    ADMUX = (ADMUX & 0b01111111) | 0b01000000;  // REFS1 = 0, REFS0 = 1: Vref = AVcc (5 V)
    ADMUX = ADMUX & 0b11111000;                 // MUX = 000: channel ADC0
    ADCSRA = ADCSRA | 0b00000111;               // ADPS = 111: ADC clock = F_CPU/128
    ADCSRA = ADCSRA | 0b10000000;               // ADEN = 1: enable the ADC
    while (1)
    {
        ADCSRA = ADCSRA | 0b01000000;           // ADSC = 1: start a conversion
        while (ADCSRA & 0b01000000);            // wait while ADSC is still 1
        float vIn = ADCW * 5.0 / 1024;          // number -> volts
        PORTB = PORTB & 0b11100000;             // all 5 LEDs off
        if      (vIn > 4) PORTB = PORTB | 0b00011111;   // 5 LEDs
        else if (vIn > 3) PORTB = PORTB | 0b00001111;   // 4 LEDs
        else if (vIn > 2) PORTB = PORTB | 0b00000111;   // 3 LEDs
        else if (vIn > 1) PORTB = PORTB | 0b00000011;   // 2 LEDs
        else if (vIn > 0) PORTB = PORTB | 0b00000001;   // 1 LED
    }
    return 0;
}
```

Line	Meaning
`(ADMUX & 0b01111111) | 0b01000000`	Clear bit 7 (REFS1) then set bit 6 (REFS0): REFS = 01 = AVcc. The other bits are kept.
`ADMUX & 0b11111000`	Clear MUX2..0, so channel 000 = ADC0 = A0.
`ADCSRA | 0b00000111`	Prescaler /128 → 125 kHz ADC clock.
`ADCSRA | 0b10000000`	Turn the ADC on. Set up first, enable last.
`ADCSRA | 0b01000000`	Start one conversion. It must be restarted for every reading.
`while (ADCSRA & 0b01000000);`	**Polling the ADC**: the empty loop spins while ADSC is 1 (~104 µs), then falls through when the hardware clears it.
`ADCW * 5.0 / 1024`	Vin = ADC × Vref / 1024. Write **5.0**, not 5: at least one number must be floating point, or `/1024` is integer division and throws the fraction away.

`PORTB & 0b11100000`	Turn the 5 LEDs off before lighting the right number, otherwise LEDs from a higher reading would stay on.

Labelled constants version: `ADMUX = (1<<REFS0); ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);` then `ADCSRA |= (1<<ADSC); while (ADCSRA & (1<<ADSC));`. Hex: `ADMUX = 0x40; ADCSRA = 0x87;`.

The Arduino version

```
void setup()
{
  pinMode(A0, INPUT);                       // not strictly needed
  for (int pin = 8; pin <= 12; pin++) pinMode(pin, OUTPUT);
}
void loop()
{
  int value = analogRead(A0);               // 0..1023 (does ADMUX/ADCSRA/ADSC for you)
  float vIn = value * 5.0 / 1024;
  for (int pin = 8; pin <= 12; pin++) digitalWrite(pin, LOW);
  int n = 0;                                // last pin to light
  if      (vIn > 4) n = 12;
  else if (vIn > 3) n = 11;
  else if (vIn > 2) n = 10;
  else if (vIn > 1) n = 9;
  else if (vIn > 0) n = 8;
  for (int pin = 8; pin <= n; pin++) digitalWrite(pin, HIGH);
}
```

6. System 9: `voltMeterUsingInterrupts`

Instead of waiting in a loop for ~104 µs every reading, let the ADC **interrupt** when the result is ready. Main is then free for another task.

```
#include <avr/io.h>
#include <avr/interrupt.h>

int main(void)
{
    DDRB |= 0b00011111;                 // LEDs
    DDRC &= 0b11111110;                 // ADC0 input
    ADMUX = (ADMUX & 0b01111111) | 0b01000000;   // Vref = AVcc
    ADMUX &= 0b11111000;                // ADC0
    ADCSRA |= 0b00000111;               // /128
    ADCSRA |= 0b10000000;               // ADEN
    ADCSRA |= 0b00001000;               // ADIE: interrupt when a conversion completes
    sei();                              // global interrupts on
    ADCSRA |= 0b01000000;               // start the FIRST conversion
    while (1) { }                       // free for another task
}

ISR(ADC_vect)                           // a result is ready
{
    float vIn = ADCW * 5.0 / 1024;
    PORTB &= 0b11100000;
    if      (vIn > 4) PORTB |= 0b00011111;
    else if (vIn > 3) PORTB |= 0b00001111;
    else if (vIn > 2) PORTB |= 0b00000111;
    else if (vIn > 1) PORTB |= 0b00000011;
    else if (vIn > 0) PORTB |= 0b00000001;
    ADCSRA |= 0b01000000;               // start the NEXT conversion
}
```

The cycle: main starts conversion 1 → ~104 µs later ADIF is set → `ISR(ADC_vect)` runs (ADIF cleared automatically), uses the result and starts conversion 2 → and so on forever. Forget the last line and the ISR runs only once.

7. Analog sensors

LDR (light dependent resistor)

Its resistance **falls** as light increases: roughly a few hundred Ω to ~1 kΩ in bright light, 100 kΩ or more in the dark (check the VT900 datasheet). The ADC cannot measure resistance, so put the LDR in a **voltage divider** with a fixed 10 kΩ.

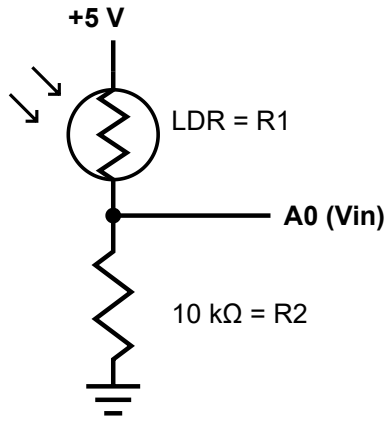


Figure 5. LDR on top: more light → LDR smaller → Vin higher.

Light	LDR (R1)	Vin = 5×10/(R1+10)	ADC
bright	1 kΩ	4.55 V	930
room	10 kΩ	2.50 V	512
dark	100 kΩ	0.45 V	93

Swap the LDR and the 10 kΩ and everything reverses (dark gives a high reading). Either is fine; just know which way your code expects.

LM35 temperature sensor

An IC with 3 legs: **+Vs, Vout, GND** (flat face towards you, left to right). It outputs a voltage **linearly proportional to temperature: 10 mV per °C**, from 0 V at 0 °C. Range −55 to +150 °C (negative temperatures need an extra circuit; with the basic connection it reads from about +2 °C).

LM35 FORMULAS

Temperature (°C) = Vout × 100 (because 1 °C = 0.01 V)
With Vref = 5 V: **T = ADC × 5.0 / 1024 × 100 = ADC × 0.488**
Example: ADC = 62 → Vout = 0.303 V → **30.3 °C**.
Resolution: one step = 4.88 mV = **0.49 °C**. With the internal 1.1 V reference: 1.07 mV = 0.11 °C per step (but max 110 °C).

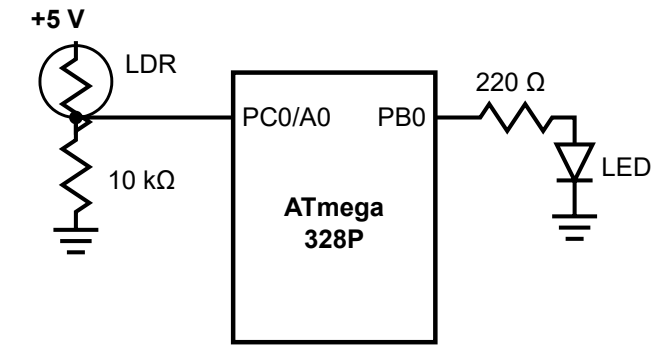
Readings jump by ±1 step? Normal: the true voltage sits on the edge of an interval and noise moves it. Take the average of several readings: `sum += analogRead(A0)` 8 times, then divide by 8.

Other analog sensors (Lesson 6 covers them)

Potentiometer, thumb joystick (two pots), accelerometer (3 voltages), water level, soil moisture, gas, sound: all give a voltage that the ADC reads exactly the same way.

8. Exercise 4 solved: the "auto light"

Spec: turn an LED on at nightfall and off in the morning light, using a light sensor. It shows the full chain: **light → LDR resistance → voltage (divider) → number (ADC) → decision (CPU) → LED (actuator).**

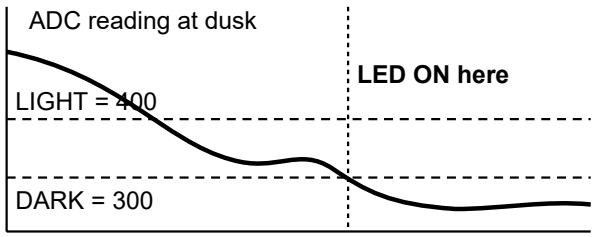


(supply, reset and crystal as in Lesson 3, P4; on the Uno they a

Figure 6. Auto light schematic.

Hysteresis: two thresholds, not one

At dusk the light changes slowly and the reading wobbles around a single threshold, so the LED would **flicker** on and off. Use **two** thresholds: switch ON below DARK, switch OFF only above LIGHT. Between them, keep the current state.



Choose the numbers by **calibrating**: print `analogRead(A0)` to the Serial Monitor (component test 04) in daylight and with the sensor covered, then pick DARK and LIGHT between those values.

Firmware, Arduino

```
const int ldrPin = A0, ledPin = 8;
const int DARK = 300, LIGHT = 400;  // calibrate!

void setup()
{
  pinMode(ledPin, OUTPUT);
}
void loop()
{
  int v = analogRead(ldrPin);
  if (v < DARK)
    digitalWrite(ledPin, HIGH);     // night: on
  else if (v > LIGHT)
    digitalWrite(ledPin, LOW);      // day: off
  delay(100);                       // between: unchanged
}
```

Firmware, C

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>
#define DARK  300
#define LIGHT 400

static unsigned int readADC0(void)
{
    ADCSRA |= (1<<ADSC);           // start
    while (ADCSRA & (1<<ADSC));    // wait
    return ADCW;
}
int main(void)
{
    DDRB |= (1<<PB0);              // LED
    DDRC &= ~(1<<PC0);             // A0 input
    ADMUX  = (1<<REFS0);           // AVcc, ADC0
    ADCSRA = (1<<ADEN) | (1<<ADPS2)
           | (1<<ADPS1) | (1<<ADPS0);  // on, /128
    while (1)
    {
        unsigned int v = readADC0();
        if (v < DARK)       PORTB |=  (1<<PB0);
        else if (v > LIGHT) PORTB &= ~(1<<PB0);
        _delay_ms(100);
    }
}
```

This is my own worked answer (the exercise has no model answer). Comparing the raw ADC number with thresholds is simpler than converting to volts first; both are correct. `ADMUX = (1<<REFS0)` sets the whole register at once: REFS = 01, ADLAR = 0, MUX = 0000.

Question 4: controlling a 230 V lamp

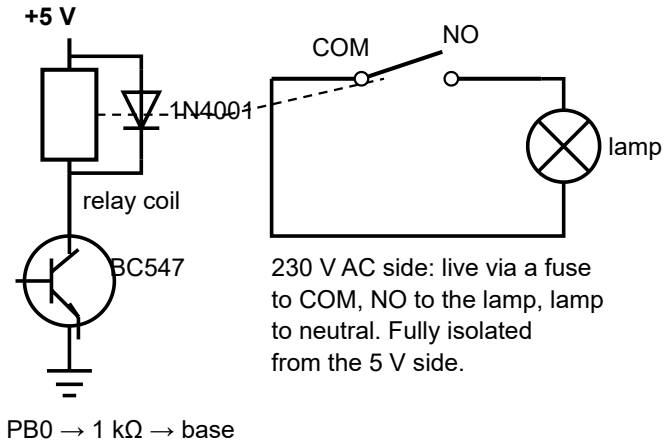


Figure 7. A relay lets 5 V logic switch mains safely (drawn for the exam; do not build it).

Why a relay? A pin gives 5 V and ~20 mA; a lamp needs 230 V AC. A **relay** is an electromagnet (coil) that closes a separate switch (contacts). The coil side is low voltage; the contact side is mains; they are **electrically isolated**.

Why a transistor? A relay coil needs more current (often 50–100 mA) than a pin can give. The pin drives a BC547 through 1 kΩ; the transistor switches the coil current (Lesson 6).

Why the diode? When the coil switches off it produces a large reverse voltage spike that can destroy the transistor. The 1N4001 across the coil (stripe to +5 V) gives that energy a safe path: the **flyback diode**.

Firmware: unchanged. PB0 HIGH now energises the relay instead of lighting an LED.

9. Building and testing the auto light

Use the method from Lesson 3, section P. Two new points for analog circuits:

1. **Divider wiring:** 5 V rail → LDR → one column; that same column → 10 kΩ → GND rail; and a jumper from that column to **A0**. The A0 wire must join the point **between** the two resistors (component test 04 used exactly this).
2. **Calibrate first, then decide:** upload a 3-line sketch that prints `analogRead(A0)` to the Serial Monitor, note the readings in light and covered, set DARK and LIGHT between them, then upload the real firmware. Test by covering the LDR with your hand: LED on; uncover: LED off, with no flicker in between.

L. What the lecturer explained in the lectures (2020 recordings)

From the five ADC recordings (folders 13–17 in `Embedded Systems Lectures`). **[EXAM]** marks things he tied to the exam.

L1. WHAT HE EXPECTS YOU TO KNOW [EXAM]

The learning outcome says **describe how the ADC converts** an analog input into a number, "not how to use the ADC. How to use it is simple; many people know. But how the conversion happens, you should be able to explain." He also said many people multiply by 5/1024 "**blindly, without any clue**": you must be able to say *why* (sections 2 and L4).

L2. Why an ADC (his words)

- Everything in the physical world varies **continuously** (temperature, light, distance, pressure, speed, acceleration): this is **analog**. Inside a computer everything is **discrete** (1s and 0s): **digital**.
- An analog sensor converts one form of energy into **electrical energy (a voltage)**: an LM35 turns thermal energy into a voltage, an LDR turns light (radiation energy) into a voltage, and a potentiometer turns rotation (kinetic energy) into a voltage. The CPU can only process digital data, so the **ADC converts the voltage into a number**.
- The input range follows the chip's operating voltage: a 5 V chip measures 0–5 V. A 10-bit ADC gives 0–1023; an 8-bit ADC on a 3.3 V chip would give 0–255 for 0–3.3 V (1.65 V → about 128). The range can be **narrowed** (e.g. 0–1.1 V) by changing the reference voltage.
- The ATmega328P has **one** ADC with **six input channels** (ADC0–ADC5 = A0–A5 = PC0–PC5). Because there is only one converter, **only one channel can be converted at a time**. Some chips have several ADCs and can convert in parallel.
- Port C pins are **shared**: a pin used as an ADC input cannot be a digital I/O pin at the same time, and vice versa.

L3. Successive approximation, his worked example [EXAM]

Block diagram: the **input multiplexer** picks the channel; **AVCC** is the ADC's own supply (normally equal to the chip's 5 V); the **reference multiplexer** picks Vref; the **DAC** turns the current guess into a voltage; the **comparator** outputs 1 if Vin > guess, else 0; the **conversion logic** stores each bit in the **data register** and feeds it back to the DAC. The ADMUX register controls both multiplexers.

The rule he used: first guess = Vref/2. If the comparator says 1, the next guess is previous + Vref/4; if 0, previous − Vref/4. Then ±Vref/8, ±Vref/16…: one step per bit. He explained it as "you don't know Vin but you know Vref, so compare Vin with slices of Vref and come closer each time".

4-bit ADC, Vin = 3 V, Vref = 5 V (his recap example)				
Step	**Guess**	**Vin > guess?**	**Bit**	**Next guess**
1	2.5	yes	1	2.5 + 5/4 = 3.75
2	3.75	no	0	3.75 − 5/8 = 3.125
3	3.125	no	0	3.125 − 5/16 = 2.8125
4	2.8125	yes	1	(done)

Result **1001 = 9** (out of 0–15). It approximated 3 V as about 2.81 V. With 10 bits the same process runs 10 times and lands much closer to 3 V. He uploaded an Excel simulation of the 10-bit process: `Lecture Materials/05 - 10-bit ADC.xlsx`. Type any Vin and watch the bits.

MORE BITS = MORE ACCURATE, BUT SLOWER

Each bit costs one compare cycle: a 4-bit ADC needs 4 steps, 8-bit 8 steps, 12-bit 12 steps. A 12-bit ADC is the most accurate but takes the longest; an 8-bit one is quicker but coarser. The ATmega328P's 10-bit ADC is the middle ground. "Successively we come closer to the input voltage, so it is called successive approximation."

L4. The formulas, explained the way he did

1. Range 0 to Vref is mapped onto 0 to $2^{\text{resolution}} - 1$, i.e. 0–1023. **0 to 1023 is 1024 values** (like 0 to 2 is 3 values: 0, 1, 2).
2. **Step size** = voltage range / number of steps = (Vref − 0) / $2^{\text{resolution}}$ = **Vref / 1024**.
3. **Vin = number of steps × step size = ADCW × Vref / 1024**. In Arduino: `analogRead(A0) * 5.0 / 1024`. Now you know why that line is written that way.
4. Put "number of steps = 1": Vin = step size. So the **step size is the smallest voltage (or change) the ADC can detect**: 5 / 1024 = **4.88 mV**; with the internal 1.1 V reference, 1.1 / 1024 = **1.07 mV**.
5. You **cannot put more than Vref** into the ADC (a student asked about 5.1 V): raise Vref first (never above the supply).

HIS ACCURACY EXAMPLE [EXAM]

A sensor outputs 0–1 V. At 501 mV and then 503 mV (a 2 mV change): with Vref = 5 V (step 4.88 mV) the ADC gives the **same number** for both, so the change is lost. With Vref = 1.1 V (step 1.07 mV) it gives **two different numbers**. **Resolution is fixed by the chip, but Vref you can choose, so choose it just above your sensor's maximum.** For a 0–3 V sensor, put 3 V on AREF and select external. He said even some industry projects leave the default 5 V with a 0–1.5 V sensor and lose accuracy.

Two sensors with different references? Possible: set the reference (`analogReference` / REFS bits), read sensor 1, change the reference, read sensor 2. External AREF can be at most the supply voltage (5 V).

L5. The registers: extra points

- **Why two data registers?** The result is 10 bits but registers are 8 bits. ADCL holds the low 8 bits; ADCH holds the top 2 bits (with six leading zeros). Combining them by hand is awkward, so the C compiler offers `ADCW`, a **virtual 16-bit register** that does it for you. It is **not in the datasheet** (an 8-bit chip has no 16-bit hardware register) and is **not available in assembly**, where you read ADCL and ADCH yourself.
- **ADSC** (start conversion): you set it to start; the hardware **clears it automatically** when the result is ready. So `while (ADCSRA & (1<<ADSC));` is **polling** the ADC.
- **ADIF** is set when the conversion is over and the data registers are updated; **ADIE** lets that raise an interrupt. All ADCSRA bits are 0 after reset, and the ADC is off (ADEN = 0) **to save power**.
- **Prescaler:** the ADC is slow (successive approximation), so it can't run at the CPU clock. ADPS = 000 is "reserved": **you must choose a prescaler**. The datasheet says **below 200 kHz** for full accuracy: 16 MHz / 128 = 125 kHz. With an 8 or 4 MHz CPU you would pick a smaller divider. If speed matters more than accuracy you may run faster than 200 kHz; the result is still reasonable, just less precise.
- **Arduino fixes the prescaler at 128**, and `analogRead` uses **polling**. Arduino has **no ADC-interrupt wrapper**: another example of Arduino being inefficient. In C you control both.
- ADMUX MUX bits: default 000 = ADC0 (A0). With several sensors on different channels: select the channel, then read; select the next, then read.

L6. Hardware he stressed in System 8

- The ADC has its **own supply pins**: **AVCC** (pin 20) to 5 V and its GND (pin 22). They must be connected on a bare chip (the Uno does it for you).
- **Potentiometer** = "potential meter": a **variable resistor**. A **wiper** slides on a circular resistive track, so the knob splits it into R1 (to 5 V) and R2 (to GND), forming a **voltage divider**: Vin = V × R2 / (R1 + R2). Fully clockwise R1 = 0, so Vin = 5 V; fully anticlockwise R2 = 0, so Vin = 0 V. Uses: volume controls, light dimmers, and the **accelerator pedal of electric and hybrid cars**.
- **One resistor for all 5 LEDs?** (a student's idea) **No.** With three LEDs at 15 mA, one shared quarter-watt resistor would carry 45 mA and could **burn**, and LEDs of different colours draw different currents. Use a **separate resistor per LED** (or a **resistor array**: one package with several resistors).
- Configuring PC0 as input (`DDRC &= ~(1<<0)`) is not required (pins are inputs after reset) but is good practice for readability, at the cost of one cycle.

L7. Code points

- **Integer-division trap [EXAM habit]:** `ADCW * 5 / 1024` is all integers, so the fraction is **thrown away**. Write `5.0` so at least one operand is floating point (implicit conversion, "casting").
- Writing a **decimal** to a port is fine: `PORTB = 3;` stores `0b00000011` (registers only ever hold bits; binary, hex and decimal are just ways of writing the number).
- Arduino version trick: store the **highest pin to light** in a variable `n` (8–12), then `for (pin = 8; pin <= n; pin++) digitalWrite(pin, HIGH);` instead of many repeated lines.
- **Interrupt-driven ADC (System 9):** polling a slow ADC wastes the CPU, so let it interrupt instead. Start the first conversion **before** the loop and **restart it at the end of** `ISR(ADC_vect)`, because ADSC is cleared after each conversion. The main loop is then free for other work.
- Binary, hex and labelled-constant versions compile to the **same** code (no difference in size or speed). Hex and labels matter on 32-bit and 64-bit chips.

L8. The two exercises he set at the end (practise these)

LIGHT METER (LDR, 4 LEDS)

LDR from 5 V to A0, 10 kΩ from A0 to GND (LDR = R1). Dark: R1 → ∞, Vin = 0. Bright: R1 → 0, Vin = 5 V. The LDR's resistance vs light curve is **not linear**. Show the light level as a percentage:
reading < 256 → 1 LED (25%)
< 512 → 2 LEDs (50%)
< 768 → 3 LEDs (75%)
up to 1023 → 4 LEDs (100%)
Write it **in C and in Arduino**.

THERMOMETER (LM35, 4 LEDS)

Flat side facing you: **left = +5 V, middle = Vout, right = GND**. Reversing + and GND **burns it out**. Temperature = Vout × 100, and Vout = ADCW × 5.0 / 1024.
below 20 °C → 1 LED
20–25 °C → 2 LEDs
25–30 °C → 3 LEDs
above 30 °C (he said 30–35) → 4 LEDs
Test by **wrapping your fingers round the sensor** (body heat about 35–36 °C).

```
// Light meter, C (my worked version; adjust the LED pins to your wiring)
#include <avr/io.h>
int main(void)
{
    DDRB  |= 0x0F;                                  // PB0-PB3: 4 LEDs
    ADMUX  = (1<<REFS0);                            // AVcc reference, channel ADC0
    ADCSRA = (1<<ADEN)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0); // on, /128
    while (1)
    {
        ADCSRA |= (1<<ADSC);                        // start
        while (ADCSRA & (1<<ADSC));                 // wait
        unsigned int v = ADCW;
        unsigned char leds;
        if      (v < 256) leds = 0x01;              // 25 %
        else if (v < 512) leds = 0x03;              // 50 %
        else if (v < 768) leds = 0x07;              // 75 %
        else              leds = 0x0F;              // 100 %
        PORTB = (PORTB & 0xF0) | leds;
    }
}
```

10. Common mistakes

Mistake	Result / fix
Forgot ADEN	ADSC never clears, so `while (ADCSRA & (1<<ADSC));` hangs forever.
Started the conversion once outside the loop	The value never updates. Set ADSC for every reading (or in the ISR).
Integer maths: `ADCW * 5 / 1024`	Result is 0, 1, 2, 3 or 4 only. Write `5.0`.
Dividing by 1023	Small error; the exam expects 1024.
A0 wire on the wrong end of the divider	Always 0 or always 1023. It must be the middle point.
Nothing on the analog pin	A floating input reads random values (handy only as a random seed, Exercise 5).
Wrong reference with a small signal	LM35 on 5 V reference: only 0.49 °C steps. Use 1.1 V if more precision is asked for.
One threshold	LED flickers at the switching point. Use two (hysteresis).
No `sei()` / no restart in `ISR(ADC_vect)`	The ISR never runs, or runs only once.

Cheat sheet: what is NOT on the exam sheets

Numbers and formulas	
Codes	2^{10} = 1024 intervals, ADC = 0..1023
Step	ΔV = Vref / 1024 (4.88 mV at 5 V)
To number	ADC = Vin × 1024 / Vref
To volts	Vin = ADC × Vref / 1024
Divider	Vout = Vs × R2 / (R1 + R2), R2 to GND
LM35	T = Vout × 100 = ADC × 0.488 (5 V ref)
ADC clock	50–200 kHz: 16 MHz / 128 = 125 kHz
Time	~13 ADC clocks ≈ 104 µs per reading

Setup recipe (polling)	
1	`ADMUX = (1<<REFS0);` AVcc, ADC0
2	`ADCSRA = (1<<ADEN)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0);`
3	`ADCSRA |= (1<<ADSC);` start
4	`while (ADCSRA & (1<<ADSC));` wait
5	`value = ADCW;`
Interrupt version adds	
	`ADCSRA |= (1<<ADIE); sei();` start once, read `ADCW` in `ISR(ADC_vect)`, restart ADSC there
Other pin	ADMUX low bits = channel (A3 → 0011)